PYTHON PROGRAMMING

Practice Handbook

Devaashi Institute

What's Inside:

- ☑ Top 30 Python Programs with Full Solutions
- ☑ Algorithm + Flowchart + Code Explanation
- ☑ Sample Input/Output for Every Program
- ☑ Time Complexity & Real-World Use Cases
- ☑ Viva Questions & Answers for Every Topic
- ☑ 10 Mini Projects with Complete Code
- ☑ Top 50 Python Viva Q&A
- ☑ Top 20 Interview Questions
- ☑ Common Errors Guide + Python Cheat Sheet
- ☑ 30-Day Practice Roadmap

Designed For:

- BCA / B.Tech / Diploma Students
- Beginner Python Programmers
- Practical Exam Preparation
- Viva & Interview Readiness
- Self-Study & Assignments

Handbook Stats:

- 30 Complete Programs
- 10 Mini Projects
- 50+ Viva Questions
- 20 Interview Questions
- Python 3 Compatible
- 50+ Pages Content

SECTION 1: BASIC PROGRAMS

Questions 1-10: Fundamentals of Python

Q1 Print Hello World

Objective

Write a Python program to display the message 'Hello, World!' on the screen.

Concept	print() is Python's built-in output function. It sends text to standard output (console). Strings can be enclosed in single or double quotes.

Algorithm (Step-by-Step)

Step 1: Start the program

Step 2: Use the print() function with the message 'Hello, World!'

Step 3: Program ends

Flow Explanation

Python reads the print() statement → sends the string argument to console → outputs text → program terminates normally.

Python Code

```
# Program 1: Hello World
# This is the simplest Python program

print('Hello, World!')    # Output the greeting message
```

Sample Input	Sample Output
(No input required)	Hello, World!

Code Explanation (Line by Line)

Line 1-2: Comments using # symbol — Python ignores these lines

Line 4: print() built-in function takes a string argument and displays it to console

The string 'Hello, World!' is passed as an argument inside parentheses

Time Complexity	Real-World Use Case
O(1) — Constant time. Single operation regardless of any input.	Used in every program as a diagnostic 'sanity check'. Printing status messages in applications, logging output to terminals.

Viva Questions & Answers

Q: What is print() in Python?

A: print() is a built-in function in Python used to output data to the standard output device (console).

Q: Can we use double quotes instead of single quotes in print()?

A: Yes. print("Hello, World!") works identically. Python accepts both single and double quotes for strings.

Q: What is a comment in Python?

A: A comment starts with # and is ignored by the Python interpreter. It is used to explain code for human readers.

Q: Is Python case-sensitive?

A: Yes. 'print' and 'Print' are different — only lowercase print() is the valid built-in function.

Alternative Method

```
# Alternative using f-string (Python 3.6+)
msg = 'Hello, World!'
print(f'{msg}')

# Using sys.stdout
import sys
sys.stdout.write('Hello, World!\n')
```

Q2 Add Two Numbers

Objective

Accept two numbers from the user and display their sum.

Concept	input() reads user input as a string. int() or float() converts strings to numbers. The + operator performs arithmetic addition.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Read first number from user (num1)

Step 3: Read second number from user (num2)

Step 4: Compute sum = num1 + num2

Step 5: Display sum

Step 6: Stop

Flow Explanation

User enters two values → input() reads as strings → int() converts to integers → addition performed → result printed.

Python Code

```
# Program 2: Add Two Numbers

# Read two numbers from the user
num1 = int(input('Enter first number: '))   # Convert input string to integer
num2 = int(input('Enter second number: '))  # Convert input string to integer

# Calculate the sum
total = num1 + num2

# Display the result
print(f'Sum of {num1} and {num2} = {total}')
```

Sample Input	Sample Output
Enter first number: 15 Enter second number: 27	Sum of 15 and 27 = 42

Code Explanation (Line by Line)

Line 4: input() displays prompt and reads user input; int() converts the string to integer

Line 5: Same for second number

Line 8: + operator adds two integers and stores result in 'total'

Line 11: f-string (formatted string literal) embeds variable values inside {}

Time Complexity	Real-World Use Case

O(1) — Single addition operation regardless of input size.	Calculator apps, billing systems, shopping cart totals, financial applications.

Viva Questions & Answers

Q: Why do we use int(input())?

A: input() always returns a string. int() converts that string to an integer so arithmetic operations can be performed.

Q: What is an f-string?

A: An f-string (formatted string literal) prefixed with f allows embedding expressions inside {} directly in a string. Introduced in Python 3.6.

Q: What happens if the user enters a letter instead of a number?

A: Python raises a ValueError because int() cannot convert a non-numeric string to an integer.

Q: How would you add two float numbers?

A: Use float(input()) instead of int(input()) to handle decimal numbers.

Alternative Method

```
# Using float for decimal support
a = float(input('Enter first: ')) b = float(input('Enter second: ')) print('Sum =', a + b)
```

Q3 Find Area of Rectangle

Objective

Calculate and display the area of a rectangle given its length and breadth.

Concept	Area of rectangle = length × breadth. The * operator performs multiplication. Variables store numeric values for computation.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input length of rectangle
Step 3: Input breadth of rectangle
Step 4: Compute area = length × breadth
Step 5: Display area
Step 6: Stop

Flow Explanation

Read length → Read breadth → Multiply both values → Store in area variable → Print result.

Python Code

```
# Program 3: Area of Rectangle

# Accept dimensions from user
length = float(input('Enter length of rectangle: '))
breadth = float(input('Enter breadth of rectangle: '))

# Calculate area using formula: Area = length * breadth
area = length * breadth

# Display result with 2 decimal places
print(f'Area of Rectangle = {area:.2f} square units')
```

Sample Input	Sample Output
Enter length: 8.5 Enter breadth: 4.0	Area of Rectangle = 34.00 square units

Code Explanation (Line by Line)

Line 4-5: float() used to accept decimal dimensions like 8.5, 4.0
Line 8: * operator multiplies length and breadth
Line 11: :.2f inside {} formats number to 2 decimal places

Time Complexity	Real-World Use Case
O(1) — Single multiplication, constant time.	Construction planning, room flooring estimation, agriculture land measurement, graphics rendering.

Viva Questions & Answers

Q: Why use float() instead of int() for dimensions?

A: Dimensions can be decimal values (e.g., 8.5 cm). float() accepts both whole and decimal numbers, making the program more general.

Q: What does :.2f mean in an f-string?

A: It formats a float number to exactly 2 decimal places. : starts the format spec, .2 means 2 decimal places, f means fixed-point float.

Q: What is the formula for the perimeter of a rectangle?

A: Perimeter = 2 × (length + breadth)

Alternative Method

```
# Compact version
l, b = float(input('Length: ')), float(input('Breadth: ')) print(f'Area =
{l*b:.2f}')
```

Q4 Find Area of Circle

Objective

Calculate the area of a circle given its radius using the formula Area = π × r².

Concept	Python's math module provides the value of π (pi). The ** operator computes powers. Import statements load external modules.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Import math module
Step 3: Input radius
Step 4: Compute area = math.pi × radius²
Step 5: Display area
Step 6: Stop

Flow Explanation

Import math → Read radius → Apply formula with math.pi and ** operator → Display formatted result.

Python Code

```
# Program 4: Area of Circle

import math   # Import math module for pi constant

# Input radius from user
radius = float(input('Enter radius of circle: '))

# Formula: Area = pi * r^2
area = math.pi * radius ** 2

# Display result rounded to 2 decimal places
print(f'Area of Circle = {area:.2f} square units')
```

Sample Input	Sample Output
Enter radius of circle: 7	Area of Circle = 153.94 square units

Code Explanation (Line by Line)

Line 3: import math loads the math standard library module
Line 6: Read radius as floating-point number
Line 9: math.pi is the constant π ≈ 3.14159...; ** is the exponentiation operator
Line 12: Result formatted to 2 decimal places using :.2f

Time Complexity	Real-World Use Case

O(1) — Simple formula evaluation, constant time.	Engineering designs, pizza delivery area calculation, satellite dish coverage, circular track measurement.

Viva Questions & Answers

Q: What is the math module?

A: The math module is a Python standard library module that provides mathematical functions and constants like math.pi, math.sqrt(), math.ceil(), etc.

Q: What does ** mean in Python?

A: ** is the exponentiation (power) operator. radius ** 2 means radius raised to the power of 2.

Q: How do you import only pi without importing entire math module?

A: Use: from math import pi — then use pi directly instead of math.pi.

Alternative Method

```
# Without math module — define pi manually
PI = 3.14159265358979 radius = float(input('Radius: ')) print(f'Area = {PI *
radius**2:.2f}')
```

Q5 Swap Two Numbers

Objective

Swap the values of two variables and display the result before and after swapping.

Concept	Python supports tuple unpacking which allows swapping variables in a single line without a temporary variable. Traditional swap uses a temp variable.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input values for a and b

Step 3: Display original values

Step 4: Swap: a, b = b, a (Python tuple swap)

Step 5: Display swapped values

Step 6: Stop

Flow Explanation

Store values → Print before → Python evaluates right side (b,a) as tuple → Unpacks into (a,b) → Values exchanged → Print after.

Python Code

```
# Program 5: Swap Two Numbers

# Input two numbers
a = int(input('Enter first number: '))
b = int(input('Enter second number: '))

# Display before swap
print(f'Before swap: a = {a}, b = {b}')

# Swap using Python tuple unpacking (Pythonic way)
a, b = b, a

# Display after swap
print(f'After swap:  a = {a}, b = {b}')
```

Sample Input	Sample Output
Enter first number: 10 Enter second number: 25	Before swap: a = 10, b = 25 After swap: a = 25, b = 10

Code Explanation (Line by Line)

Lines 4-5: Read two integers from user

Line 8: Display original values using f-string

Line 11: a, b = b, a — Python evaluates right side first as tuple (b, a), then unpacks into a and b

Line 14: Display swapped values confirming the exchange

Time Complexity	Real-World Use Case
O(1) — Simple assignment operations.	Sorting algorithms (bubble sort swaps), card game shuffling, data rearrangement in databases.

Viva Questions & Answers

Q: How does Python swap without a temp variable?

A: Python uses tuple packing and unpacking. The right side b,a creates a tuple; the left side a,b unpacks it — all in one atomic operation.

Q: Show the traditional swap using a temp variable.

A: temp = a; a = b; b = temp — uses a third variable to temporarily hold one value.

Q: Can you swap strings in Python the same way?

A: Yes! Python's tuple swap works for any data type: x, y = y, x works for strings, floats, lists, etc.

Alternative Method

```
# Traditional method with temp variable
temp = a a = b b = temp print(f'Swapped: a={a}, b={b}')

# Using XOR (integers only)
a = a ^ b b = a ^ b a = a ^ b
```

Q6 Check Even or Odd

Objective

Determine whether a given number is even or odd using the modulus operator.

Concept	The modulus operator % gives the remainder of division. If n % 2 == 0, the number is even; otherwise it is odd. Conditional if-else controls program flow.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input a number n

Step 3: If n % 2 == 0, print 'Even'

Step 4: Else print 'Odd'

Step 5: Stop

Flow Explanation

Read number → Apply modulus 2 → If remainder is 0: Even → Else: Odd → Display result.

Python Code

```
# Program 6: Check Even or Odd

# Read input from user
n = int(input('Enter a number: '))

# Check using modulus operator
if n % 2 == 0:            # % gives remainder; 0 means divisible by 2
    print(f'{n} is Even') # Indented block executes if condition is True
else:
    print(f'{n} is Odd')  # Executes if condition is False
```

Sample Input	Sample Output
Enter a number: 14	14 is Even

Code Explanation (Line by Line)

Line 4: int(input()) reads and converts the number

Line 7: n % 2 computes remainder when n is divided by 2

Line 8: If remainder == 0, number is exactly divisible by 2 → Even

Lines 9-10: else block executes when remainder is 1 → Odd

Time Complexity	Real-World Use Case
O(1) — Single modulus check and comparison.	Distributing items in pairs, alternating row colors in tables, parity checks in networking.

Viva Questions & Answers

Q: What is the modulus operator?

A: % returns the remainder of integer division. Example: 7 % 2 = 1, 8 % 2 = 0.

Q: Is 0 even or odd?

A: 0 is even because 0 % 2 == 0.

Q: How would you check if a number is divisible by 5?

A: Use n % 5 == 0. If True, the number is divisible by 5.

Q: What is the difference between == and =?

A: = is assignment (stores a value); == is comparison (checks equality and returns True/False).

Alternative Method

```
# Using bitwise AND
if n & 1 == 0:     print('Even') else:     print('Odd')

# One-liner using ternary
result = 'Even' if n % 2 == 0 else 'Odd' print(result)
```

Q7 Find Largest of Two Numbers

Objective

Compare two numbers and display the larger one.

Concept	Comparison operators (>, <, ==) compare values. if-elif-else provides multi-branch decision making. Conditional (ternary) expressions offer a concise alternative.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input numbers a and b

Step 3: If a > b, print a is larger

Step 4: Elif a < b, print b is larger

Step 5: Else print both are equal

Step 6: Stop

Flow Explanation

Read a and b → Compare a > b → True: a is max → False: compare a < b → True: b is max → False: equal.

Python Code

```
# Program 7: Largest of Two Numbers

# Accept two numbers
a = float(input('Enter first number: '))
b = float(input('Enter second number: '))

# Compare and find largest
if a > b:
    print(f'{a} is the largest number')
elif a < b:
    print(f'{b} is the largest number')
else:
    print(f'Both numbers are equal ({a})')
```

Sample Input	Sample Output
Enter first number: 45 Enter second number: 32	45.0 is the largest number

Code Explanation (Line by Line)

Lines 4-5: Read both numbers as floats to handle decimals

Line 8: if a > b — first condition checks if a strictly greater than b

Line 10: elif (else-if) checks next condition only if first was False

Line 12: else catches the remaining case — numbers are equal

Time Complexity	Real-World Use Case
O(1) — Constant comparisons.	Leaderboard rankings, price comparison sites, finding highest bid in auctions.

Viva Questions & Answers

Q: What is elif?

A: elif means 'else if' — checks an additional condition only when the preceding if condition is False. You can chain multiple elif blocks.

Q: How does Python's built-in max() work?

A: max(a, b) returns the larger of the two values. It can also take a list: max([3,7,2]) returns 7.

Q: Can you use the ternary operator here?

A: Yes: largest = a if a > b else b. This assigns the larger value in one line.

Alternative Method

```
# Using Python's built-in max() largest = max(a, b) print(f'Largest = {largest}')

# One-liner ternary print(a if a > b else b)
```

Q8 Find Largest of Three Numbers

Objective

Compare three numbers and identify the largest among them.

Concept	Nested if-elif-else or logical AND (and) operators handle multiple comparisons simultaneously. Chained conditions test multiple criteria.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input a, b, c

Step 3: If a > b and a > c, a is largest

Step 4: Elif b > c, b is largest

Step 5: Else c is largest

Step 6: Stop

Flow Explanation

Compare a with both others using 'and' → Check b vs c → Remaining case is c.

Python Code

```
# Program 8: Largest of Three Numbers

# Input three numbers
a = float(input('Enter first number: '))
b = float(input('Enter second number: '))
c = float(input('Enter third number: '))

# Find largest using logical AND operator
if a >= b and a >= c:       # 'and' requires BOTH conditions to be True
    print(f'{a} is the largest')
elif b >= a and b >= c:
    print(f'{b} is the largest')
else:
    print(f'{c} is the largest')
```

Sample Input	Sample Output
Enter first number: 12 Enter second number: 48 Enter third number: 35	48.0 is the largest

Code Explanation (Line by Line)

Lines 4-6: Read three float values

Line 9: a >= b and a >= c — both conditions must be True simultaneously (using logical AND)

Line 11: elif checks if b is largest when a is not

Line 13: else — if neither a nor b is largest, c must be

Time Complexity	Real-World Use Case
O(1) — Fixed number of comparisons regardless of values.	Highest scorer in exams, maximum temperature of three cities, top salary among candidates.

Viva Questions & Answers

Q: What is the 'and' operator?

A: 'and' is a logical operator that returns True only when BOTH conditions are True. If either is False, the result is False.

Q: What is the difference between > and >=?

A: > means strictly greater than (excludes equality). >= means greater than or equal to (includes equality).

Q: How to find the largest using max()?

A: max(a, b, c) returns the largest. Even simpler: max([a, b, c]).

Alternative Method

```
# Using max() built-in
a, b, c = 12, 48, 35 print(f'Largest = {max(a,b,c)}')

# Using list
nums = [a, b, c] print(f'Largest = {max(nums)}')
```

Q9 Calculate Simple Interest

Objective

Calculate Simple Interest using the formula SI = (P × R × T) / 100.

Concept	Simple Interest is a basic financial formula. Variable assignment stores intermediate results. Float arithmetic handles decimal precision.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input principal P, rate R, time T
Step 3: Compute SI = (P * R * T) / 100
Step 4: Compute Amount = P + SI
Step 5: Display SI and Amount
Step 6: Stop

Flow Explanation

Read P, R, T → Apply formula → Calculate amount → Display both results.

Python Code

```
# Program 9: Simple Interest

# Input the required values
P = float(input('Enter Principal amount (₹): '))
R = float(input('Enter Rate of interest (%): '))
T = float(input('Enter Time period (years): '))

# Formula: SI = (P * R * T) / 100
SI = (P * R * T) / 100

# Total amount after interest
Amount = P + SI

# Display results
print(f'\nSimple Interest = ₹{SI:.2f}')
print(f'Total Amount    = ₹{Amount:.2f}')
```

Sample Input	Sample Output
Principal: 5000 Rate: 8 Time: 3	Simple Interest = ₹1200.00 Total Amount = ₹6200.00

Code Explanation (Line by Line)

Lines 4-6: Read Principal, Rate, Time as floating-point values
Line 9: Apply the SI formula — parentheses ensure correct operator precedence

Line 12: Total amount is principal plus interest earned

Lines 15-16: \n adds blank line; ₹ is the Rupee symbol; :.2f gives 2 decimal places

Time Complexity	Real-World Use Case
O(1) — Fixed arithmetic operations.	Banking systems, loan calculators, savings account interest computation, financial planning apps.

Viva Questions & Answers

Q: What is the formula for Simple Interest?

A: SI = (P × R × T) / 100 where P = Principal, R = Rate per annum, T = Time in years.

Q: What is the difference between Simple and Compound Interest?

A: Simple Interest is calculated only on the principal. Compound Interest is calculated on principal plus previously earned interest.

Q: Why divide by 100 in the SI formula?

A: Because R is given as a percentage (e.g., 8%). Dividing by 100 converts it to a decimal fraction (0.08).

Alternative Method

```
# Compound Interest alternative
n = 1  # compounded annually CI = P * (1 + R/(n*100))**(n*T) - P print(f'Compound Interest = {CI:.2f}')
```

Q10 Calculate Percentage and Grade

Objective

Calculate the percentage of marks obtained and assign the appropriate grade.

Concept	Percentage = (marks obtained / total marks) × 100. Grade assignment uses chained if-elif-else. String formatting displays results neatly.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input marks in 5 subjects

Step 3: Compute total and percentage

Step 4: Use if-elif-else to assign grade

Step 5: Display percentage and grade

Step 6: Stop

Flow Explanation

Sum 5 marks → Calculate percentage → Check grade thresholds from highest to lowest → Assign grade → Display.

Python Code

```
# Program 10: Percentage and Grade

# Input marks for 5 subjects
print('Enter marks out of 100 for each subject:')
m1 = float(input('Subject 1: '))
m2 = float(input('Subject 2: '))
m3 = float(input('Subject 3: '))
m4 = float(input('Subject 4: '))
m5 = float(input('Subject 5: '))

# Calculate total and percentage
total = m1 + m2 + m3 + m4 + m5
percentage = (total / 500) * 100   # 5 subjects × 100 marks each = 500 total

# Grade assignment based on percentage
if percentage >= 90:
    grade = 'A+ (Outstanding)'
elif percentage >= 75:
    grade = 'A  (Distinction)'
elif percentage >= 60:
    grade = 'B  (First Class)'
elif percentage >= 50:
    grade = 'C  (Second Class)'
elif percentage >= 40:
    grade = 'D  (Pass)'
else:
    grade = 'F  (Fail)'
```

```
# Display results
print(f'\nTotal Marks  : {total}/500')
print(f'Percentage   : {percentage:.2f}%')
print(f'Grade        : {grade}')
```

Sample Input	Sample Output
85, 78, 92, 88, 76	Total Marks : 419.0/500 Percentage : 83.80% Grade : A (Distinction)

Code Explanation (Line by Line)

Lines 5-9: Read marks for 5 subjects using float() for decimal marks

Line 12: Sum all five marks into total

Line 13: Percentage = (obtained/max) × 100 where max = 500

Lines 16-26: Chained elif checks thresholds from highest to lowest — ORDER MATTERS

Lines 29-31: Display formatted results

Time Complexity	Real-World Use Case
O(1) — Fixed comparisons regardless of marks values.	School/university result systems, competitive exam scorecards, performance evaluation dashboards.

Viva Questions & Answers

Q: Why must grade conditions be checked from highest to lowest?

A: Because elif stops at the first True condition. Checking >= 40 first would match all passing grades before reaching >=90.

Q: How would you handle marks entered as a list?

A: marks = [m1,m2,m3,m4,m5]; total = sum(marks); percentage = (total/500)*100.

Q: What grade system does your code use?

A: A custom percentage-based system: A+(>=90), A(>=75), B(>=60), C(>=50), D(>=40), F(<40).

SECTION 2: LOOP-BASED PROGRAMS

Questions 11-20: Loops & Number Logic

Q11 Print Numbers from 1 to N

Objective

Use a for loop to print all integers from 1 to N on separate lines.

Concept	for loop iterates over a sequence. range(1, n+1) generates integers from 1 to n inclusive. Loops execute a block repeatedly.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input N
Step 3: For i from 1 to N (inclusive)
Step 4: Print i
Step 5: End loop
Step 6: Stop

Flow Explanation

Read N → range(1,N+1) generates sequence → loop variable i takes each value → print i → repeat until N.

Python Code

```
# Program 11: Print Numbers 1 to N

# Input the upper limit
n = int(input('Enter a positive number N: '))

print(f'Numbers from 1 to {n}:')

# for loop with range
for i in range(1, n + 1):    # range(1, n+1) produces 1,2,...,n
    print(i, end=' ')         # end=' ' prints space instead of newline

print()  # Move to next line after loop
```

Sample Input	Sample Output
Enter N: 10	Numbers from 1 to 10: 1 2 3 4 5 6 7 8 9 10

Code Explanation (Line by Line)

Line 4: Read integer N
Line 9: range(1, n+1) — starts at 1, stops BEFORE n+1 (so it includes n)
Line 10: end=' ' changes default newline to space, printing all on one line

Line 12: print() without arguments adds the final newline

Time Complexity	Real-World Use Case
O(N) — Loop runs N times, linear time.	Generating numbered lists, batch processing N items, creating sequential IDs.

Viva Questions & Answers

Q: What does range(1, n+1) produce?

A: It produces a sequence: 1, 2, 3, ..., n. range() excludes the end value, so we add 1 to include n.

Q: What is the default end parameter in print()?

A: By default, end='\n' (newline). Changing to end=' ' prints items on the same line.

Q: Difference between for loop and while loop?

A: for iterates over a known sequence/range. while keeps looping while a condition is True — useful when number of iterations is unknown.

Alternative Method

```
# Using while loop
i = 1 while i <= n:     print(i, end=' ')     i += 1
```

Q12 Sum of First N Natural Numbers

Objective

Calculate the sum of all natural numbers from 1 to N using a loop and compare with the formula.

Concept	Accumulator pattern: start with sum=0, add each number in a loop. Mathematical formula: Sum = N×(N+1)/2 gives O(1) solution.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input N

Step 3: Initialize total = 0

Step 4: For i from 1 to N: total += i

Step 5: Display total

Step 6: Stop

Flow Explanation

Read N → Initialize accumulator → Each iteration adds i to total → After loop display total.

Python Code

```
# Program 12: Sum of First N Natural Numbers

n = int(input('Enter N: '))

# Method 1: Using loop (accumulator pattern)
total = 0            # Initialize sum accumulator to 0
for i in range(1, n + 1):
    total += i       # total = total + i  (shorthand)

print(f'Sum of first {n} natural numbers = {total}')

# Method 2: Using formula (instant calculation)
formula_sum = n * (n + 1) // 2   # Integer division
print(f'Using formula: N*(N+1)/2  = {formula_sum}')
```

Sample Input	Sample Output
Enter N: 100	Sum of first 100 natural numbers = 5050 Using formula: N*(N+1)/2 = 5050

Code Explanation (Line by Line)

Line 6: total=0 starts accumulator

Line 8: += is augmented assignment: total = total + i

Line 13: // is integer (floor) division — avoids float result

Time Complexity	Real-World Use Case
Loop method: O(N). Formula method: O(1).	Payroll systems (sum of n employees), statistics (summation), database aggregation.

Viva Questions & Answers

Q: What is an accumulator variable?

A: A variable initialized to a neutral value (0 for sum, 1 for product) that collects results across loop iterations.

Q: What does += mean?

A: It is augmented assignment. total += i means total = total + i. Shorthand to update a variable.

Q: What is Gauss's formula for sum of N numbers?

A: Sum = N × (N+1) / 2. Gauss discovered this at age 9 to quickly sum 1 to 100 = 5050.

Alternative Method

```
# Using Python sum() and range()
n = int(input('N: ')) print(f'Sum = {sum(range(1, n+1))}')
```

Q13 Multiplication Table of a Number

Objective

Display the multiplication table of a given number from 1 to 10.

Concept	Nested string formatting with loops. The * operator multiplies. f-strings align output neatly with padding specifiers.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input number n

Step 3: For i from 1 to 10

Step 4: Print: n × i = n*i

Step 5: End loop

Step 6: Stop

Flow Explanation

Read n → Loop i from 1..10 → Multiply n*i → Print formatted line → Repeat.

Python Code

```
# Program 13: Multiplication Table

n = int(input('Enter a number: '))

print(f'\n--- Multiplication Table of {n} ---')
print('-' * 30)  # Print a separator line

for i in range(1, 11):           # Loop from 1 to 10
    result = n * i
    print(f'{n:3} x {i:2} = {result:4}')  # Formatted alignment
```

Sample Input	Sample Output
Enter a number: 7	--- Multiplication Table of 7 --- 7 x 1 = 7 7 x 2 = 14 7 x 3 = 21 7 x 10 = 70

Code Explanation (Line by Line)

Line 6: '-' * 30 repeats the '-' character 30 times (string repetition)

Line 8: range(1, 11) gives 1 through 10

Line 10: :3 right-aligns in 3 chars, :2 in 2 chars, :4 in 4 chars — creates neat columns

Time Complexity	Real-World Use Case
O(1) — Always exactly 10 iterations.	Educational apps, math worksheets, generating lookup tables for embedded systems.

Viva Questions & Answers

Q: What does string * number do in Python?

A: String repetition: 'ab' * 3 produces 'ababab'. Used here to create separator lines.

Q: What does :3 do in an f-string?

A: It sets the minimum field width to 3 characters, right-aligning the number (padding with spaces on left).

Q: How would you print the table up to 20?

A: Change range(1, 11) to range(1, 21).

Q14 Factorial of a Number

Objective

Calculate the factorial of a non-negative integer using a loop.

Concept	Factorial n! = 1×2×3×...×n. Product accumulator starts at 1. Special case: 0! = 1. Validation prevents negative input.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input n
Step 3: Validate n >= 0
Step 4: Initialize fact = 1
Step 5: For i from 1 to n: fact *= i
Step 6: Print fact
Step 7: Stop

Flow Explanation

Read n → Validate → Loop multiplying 1×2×3×...×n → Display result.

Python Code

```
# Program 14: Factorial of a Number

n = int(input('Enter a non-negative integer: '))

# Validate input
if n < 0:
    print('Factorial is not defined for negative numbers!')
else:
    fact = 1    # Factorial accumulator — start at 1 (not 0!)

    for i in range(1, n + 1):  # Multiply 1×2×3×...×n
        fact *= i               # fact = fact * i

    print(f'{n}! = {fact}')
```

Sample Input	Sample Output
Enter a non-negative integer: 6	6! = 720

Code Explanation (Line by Line)

Lines 6-7: Guard clause — reject negative numbers immediately
Line 9: fact=1 because multiplying by 0 would always give 0
Line 11: range(1, n+1) iterates 1,2,3,...,n
Line 12: *= is shorthand: fact = fact * i

Note: When n=0, range(1,1) is empty, so loop doesn't run → fact=1 → 0!=1 ✓

Time Complexity	Real-World Use Case
O(N) — Loop runs N times.	Combinatorics (permutations/combinations), probability calculations, scientific computing.

Viva Questions & Answers

Q: What is the value of 0!?

A: 0! = 1 by mathematical definition. Our code handles this correctly — when n=0 the loop doesn't execute, so fact stays 1.

Q: What is the largest factorial Python can compute?

A: Python integers have unlimited precision, so Python can compute factorial of very large numbers (unlike C/Java which overflow).

Q: How to compute factorial using recursion?

A: def factorial(n): return 1 if n==0 else n * factorial(n-1)

Alternative Method

```
# Using math.factorial()
import math n = int(input('N: ')) print(f'{n}! = {math.factorial(n)}')

# Recursive def fact(n):     return 1 if n <= 1 else n * fact(n-1) print(fact(6))
```

Q15 Reverse a Number

Objective

Reverse the digits of a given integer without using string conversion.

Concept	Extract digits using modulus (% 10) and remove with floor division (// 10). Build reversed number by multiplying by 10 and adding digit.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input n, store original

Step 3: Initialize reversed=0

Step 4: While n > 0: digit=n%10; reversed=reversed*10+digit; n=n//10

Step 5: Print reversed

Step 6: Stop

Flow Explanation

Extract last digit → Append to reversed → Remove last digit from n → Repeat until n becomes 0.

Python Code

```
# Program 15: Reverse a Number

n = int(input('Enter a number: '))
original = n      # Save original for display
reversed_num = 0  # Will build the reversed number

while n > 0:
    digit = n % 10          # Extract the LAST digit
    reversed_num = reversed_num * 10 + digit  # Shift left and add digit
    n = n // 10             # Remove the last digit

print(f'Original  : {original}')
print(f'Reversed  : {reversed_num}')
```

Sample Input	Sample Output
Enter a number: 12345	Original : 12345 Reversed : 54321

Code Explanation (Line by Line)

Line 4: Save original since we modify n in the loop

Line 7: Loop runs while digits remain in n

Line 8: % 10 gives the units digit (12345 % 10 = 5)

Line 9: Shift reversed left (*10) then add new digit

Line 10: // 10 removes the last digit (12345 // 10 = 1234)

Time Complexity	Real-World Use Case
O(d) where d = number of digits. O(log N) in terms of N.	Palindrome number check (prerequisite), cryptography, barcode verification algorithms.

Viva Questions & Answers

Q: Why multiply reversed_num by 10 before adding the digit?

A: Multiplying by 10 shifts existing digits left (makes room). Then adding the new digit places it in the units position.

Q: What is integer division in Python?

A: // is floor division — returns the quotient without remainder. 12345 // 10 = 1234 (removes last digit).

Q: How would you reverse using strings?

A: str(n)[::-1] reverses the string representation. [::-1] is Python's reverse slice.

Alternative Method

```
# Using string slicing (simpler)
n = input('Number: ') print(f'Reversed: {n[::-1]}')
```

Q16 Count Digits in a Number

Objective

Count the total number of digits in a given positive integer.

Concept	Repeatedly dividing by 10 removes one digit at a time. A counter increments each iteration. len(str(n)) offers an alternative O(1) approach.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input n
Step 3: Initialize count=0
Step 4: While n > 0: count+=1; n //= 10
Step 5: Print count
Step 6: Stop

Flow Explanation

Remove digits one by one → Count each removal → When n=0 loop ends → Display count.

Python Code

```
# Program 16: Count Digits in a Number

n = int(input('Enter a positive integer: '))
original = n
count = 0           # Digit counter

while n > 0:
    count += 1      # Increment count for each digit
    n //= 10        # Remove the last digit (n = n // 10)

print(f'Number: {original}')
print(f'Total digits: {count}')
```

Sample Input	Sample Output
Enter a positive integer: 987654	Number: 987654 Total digits: 6

Code Explanation (Line by Line)

Line 5: count=0 initializes our counter
Line 8: count += 1 each time we process one digit
Line 9: //= 10 is shorthand for n = n // 10 — removes last digit
The loop stops when n becomes 0 (all digits removed)

Time Complexity	Real-World Use Case

O(d) where d = number of digits, equivalent to $O(\log_{10} N)$.	PIN validation (must be 4 or 6 digits), phone number validation, OTP length checks.

Viva Questions & Answers

Q: How many digits does 1000 have?

A: 4 digits. Our loop: 1000→100(1) →10(2) →1(3) →0(4) → stops. count=4 ✓

Q: What is the fastest way to count digits?

A: len(str(n)) converts to string and counts characters. O(1) practically, but involves string conversion.

Q: How would you handle the number 0?

A: 0 has 1 digit, but our while loop doesn't execute for n=0. Handle separately: if n==0: count=1.

Alternative Method

```
# Using string length (one-liner)
n = int(input('N: ')) print(f'Digits: {len(str(abs(n)))}')

# Using math.log10
import math count = int(math.log10(n)) + 1 if n > 0 else 1 print(count)
```

Q17 Sum of Digits

Objective

Calculate the sum of all individual digits of a given number.

Concept	Same digit extraction technique as counting: n%10 gives last digit, n//10 removes it. Add each digit to a running total.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input n

Step 3: Initialize digit_sum=0

Step 4: While n>0: digit_sum += n%10; n //= 10

Step 5: Print digit_sum

Step 6: Stop

Flow Explanation

Extract last digit → Add to sum → Remove digit → Repeat → Display total sum.

Python Code

```
# Program 17: Sum of Digits

n = int(input('Enter a number: '))
original = n
digit_sum = 0      # Running sum of digits

while n > 0:
    digit = n % 10          # Get last digit
    digit_sum += digit      # Add to sum
    n //= 10                # Remove last digit

print(f'Number    : {original}')
print(f'Sum of digits: {digit_sum}')
```

Sample Input	Sample Output
Enter a number: 4567	Number : 4567 Sum of digits: 22

Code Explanation (Line by Line)

4567: extract 7 (sum=7) → 456: extract 6 (sum=13) → 45: extract 5 (sum=18) → 4: extract 4 (sum=22) → 0: stop

Line 8: % 10 extracts units digit

Line 9: += adds digit to accumulator

Line 10: //= 10 removes the processed digit

Time Complexity	Real-World Use Case
O(d) — proportional to number of digits.	Digital root calculation, checksum algorithms (Luhn algorithm for credit cards), numerology apps.

Viva Questions & Answers

Q: What is the digital root of a number?

A: Keep summing digits until a single digit remains. 4567 → 22 → 4. It's the number modulo 9 (with 9 instead of 0 for multiples of 9).

Q: How to find sum of digits using string conversion?

A: sum(int(d) for d in str(n)) — convert to string, iterate characters, convert each back to int, sum all.

Q: What is the sum of digits of 999?

A: 9+9+9 = 27.

Alternative Method

```
# Using string method
n = input('Number: ') digit_sum = sum(int(d) for d in n if d.isdigit()) print(f'Sum = {digit_sum}')
```

Q18 Check Palindrome Number

Objective

Determine if a number reads the same forwards and backwards.

Concept	A palindrome number equals its reverse. Reverse the digits using the modulus technique and compare with the original.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input n, save original
Step 3: Reverse n using digit extraction
Step 4: If reversed == original: Palindrome
Step 5: Else: Not Palindrome
Step 6: Stop

Flow Explanation

Read n → Reverse digits → Compare reversed with original → Print result.

Python Code

```
# Program 18: Check Palindrome Number

n = int(input('Enter a number: '))
original = n          # Save original for comparison
reversed_num = 0

# Reverse the number
temp = n
while temp > 0:
    digit = temp % 10
    reversed_num = reversed_num * 10 + digit
    temp //= 10

# Compare original with reversed
if original == reversed_num:
    print(f'{original} is a Palindrome number ✓')
else:
    print(f'{original} is NOT a Palindrome number ✗')
```

Sample Input	Sample Output
Enter a number: 12321	12321 is a Palindrome number ✓

Code Explanation (Line by Line)

Lines 8-12: Reverse n into reversed_num (same technique as Q15)

Line 15: Compare original value with reversed — if equal, it's a palindrome
Examples: 121, 1331, 12321, 99 are palindromes; 123, 100 are not

Time Complexity	Real-World Use Case
O(d) — proportional to number of digits.	Error detection, symmetric data validation, number theory research.

Viva Questions & Answers

Q: Give 5 examples of palindrome numbers.

A: 121, 1331, 12321, 99, 10001 — all read the same left-to-right and right-to-left.

Q: Is 1 a palindrome?

A: Yes. Single-digit numbers are always palindromes.

Q: How to check palindrome using string method?

A: str(n) == str(n)[::-1] — reverse string and compare with original.

Alternative Method

```
# String method (simpler)
n = input('Enter number: ') if n == n[::-1]:     print(f'{n} is Palindrome') else:
print('Not Palindrome')
```

Q19 Check Armstrong Number

Objective

Determine if a number is an Armstrong (narcissistic) number.

Concept	An Armstrong number equals the sum of its digits each raised to the power of the number of digits. E.g., 153 = $1^3+5^3+3^3$ = 153.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input n
Step 3: Count digits d
Step 4: Compute sum of (each digit ^ d)
Step 5: If sum == n: Armstrong
Step 6: Else: Not Armstrong
Step 7: Stop

Flow Explanation

Count digits → Extract each digit → Raise to power d → Sum all → Compare with original.

Python Code

```
# Program 19: Check Armstrong Number

n = int(input('Enter a number: '))
original = n

# Count the number of digits
num_digits = len(str(n))      # e.g., 153 has 3 digits

# Compute Armstrong sum
arm_sum = 0
temp = n
while temp > 0:
    digit = temp % 10
    arm_sum += digit ** num_digits  # Each digit raised to power = no. of digits
    temp //= 10

# Check Armstrong condition
if arm_sum == original:
    print(f'{original} is an Armstrong number ✓')
    print(f'Proof: sum of digits^{num_digits} = {arm_sum}')
else:
    print(f'{original} is NOT an Armstrong number ✗')
```

Sample Input	Sample Output

Enter a number: 153	153 is an Armstrong number ✓ Proof: sum of digits^3 = 153

📖 Code Explanation (Line by Line)

Line 7: len(str(n)) counts digits by converting to string

Lines 12-15: Loop extracts each digit and raises to power num_digits

Line 18: If accumulated sum equals original → Armstrong number

153: $1^3+5^3+3^3 = 1+125+27 = 153$ ✓; 370: $3^3+7^3+0^3 = 27+343+0 = 370$ ✓

⏱ Time Complexity	🌍 Real-World Use Case
O(d) — proportional to number of digits.	Number theory education, recreational mathematics, mathematical puzzle apps.

🎓 Viva Questions & Answers

Q: List Armstrong numbers up to 500.

A: 1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407.

Q: Why is 1 an Armstrong number?

A: 1 has 1 digit, and $1^1 = 1$. So it satisfies the condition.

Q: What is the 4-digit Armstrong number?

A: 1634 ($1^4+6^4+3^4+4^4=1+1296+81+256=1634$), 8208, 9474.

🔀 Alternative Method

```
# Compact using list comprehension
n = int(input('N: ')) d = len(str(n)) if sum(int(x)**d for x in str(n)) == n:
print(f'{n} is Armstrong') else:     print('Not Armstrong')
```

Q20 Check Prime Number

Objective

Determine if a given number is prime (divisible only by 1 and itself).

Concept	A prime has exactly 2 factors: 1 and itself. Check divisibility from 2 to √n — if any factor found, not prime. √n optimization reduces iterations.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input n

Step 3: If n < 2: not prime

Step 4: For i from 2 to √n: if n%i==0: not prime, break

Step 5: If no factor found: prime

Step 6: Stop

Flow Explanation

Read n → Handle edge cases → Test divisors 2..√n → If any divides evenly → Not prime → Else → Prime.

Python Code

```
# Program 20: Check Prime Number

import math

n = int(input('Enter a number: '))

def is_prime(num):
    if num < 2:
        return False          # 0 and 1 are NOT prime
    if num == 2:
        return True           # 2 is the only even prime
    if num % 2 == 0:
        return False          # Even numbers > 2 are not prime

    # Check odd divisors from 3 to sqrt(n)
    for i in range(3, int(math.sqrt(num)) + 1, 2):
        if num % i == 0:      # If divisible, not prime
            return False
    return True               # No divisors found → prime

if is_prime(n):
    print(f'{n} is a Prime number ✓')
else:
    print(f'{n} is NOT a Prime number ✗')
```

Sample Input	Sample Output

Enter a number: 97	97 is a Prime number ✓

📖 Code Explanation (Line by Line)

Lines 8-9: Numbers < 2 are not prime by definition

Lines 10-11: 2 is special — the only even prime

Lines 12-13: Skip all even numbers in one check

Line 16: range(3, √n+1, 2) checks only odd numbers 3,5,7,... (step=2)

√n optimization: if n has a factor > √n, the corresponding factor is < √n (already found)

⏱ Time Complexity	🌍 Real-World Use Case
O(√N) — much faster than O(N).	Cryptography (RSA encryption), hash table sizing, random number generation.

🎓 Viva Questions & Answers

Q: Why do we only check up to √n?

A: If n = a×b and both a,b > √n, then a×b > n — contradiction. So at least one factor must be ≤ √n.

Q: Is 1 a prime number?

A: No. By definition, primes have exactly 2 distinct factors. 1 has only one factor (itself).

Q: What is the smallest prime number?

A: 2 is the smallest and the only even prime number.

Q: What is a composite number?

A: A composite number has more than 2 factors — i.e., it is not prime and not 1. Examples: 4, 6, 8, 9.

🔀 Alternative Method

```
# Brute force O(N)
def is_prime_bf(n):     if n < 2: return False     for i in range(2, n):         if
n % i == 0: return False     return True
```

SECTION 3: SERIES & PATTERN PROGRAMS

Questions 21-25: Series Generation & Patterns

Q21 Fibonacci Series

Objective

Generate the Fibonacci series up to N terms where each term is the sum of the two preceding terms.

Concept	Fibonacci: 0,1,1,2,3,5,8,13... Each term = sum of previous two. Uses two variables to track last two values. Series appears in nature.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input N (number of terms)

Step 3: Initialize a=0, b=1

Step 4: Print a

Step 5: For i from 2 to N: c=a+b; print c; a=b; b=c

Step 6: Stop

Flow Explanation

Initialize first two terms → Print each → Update: new term = sum of last two → Shift variables forward.

Python Code

```
# Program 21: Fibonacci Series

n = int(input('Enter number of terms: '))

a, b = 0, 1     # First two Fibonacci numbers

print(f'Fibonacci series ({n} terms):')
for i in range(n):
    print(a, end=' ')  # Print current term
    a, b = b, a + b    # Shift: a becomes b, b becomes a+b (next term)

print()  # New line at end
```

Sample Input	Sample Output
Enter number of terms: 10	Fibonacci series (10 terms): 0 1 1 2 3 5 8 13 21 34

Code Explanation (Line by Line)

Line 5: Start with first two terms a=0, b=1

Line 9: Print current term a

Line 10: Elegant simultaneous update — a,b = b,(a+b). Right side evaluated before assignment

Time Complexity	Real-World Use Case
O(N) — one iteration per term.	Financial modeling (Fibonacci retracements), image compression, natural growth patterns, UI spiral designs.

Viva Questions & Answers

Q: What are the first 10 Fibonacci numbers?

A: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

Q: Where does Fibonacci appear in nature?

A: Flower petal counts, spiral arrangements in sunflowers and pine cones, rabbit population growth model.

Q: How to compute Fibonacci using recursion?

A: def fib(n): return n if n<=1 else fib(n-1)+fib(n-2). (Inefficient — use memoization for large n.)

Alternative Method

```
# Recursive with memoization
from functools import lru_cache @lru_cache(maxsize=None) def fib(n):     return n if n <= 1 else fib(n-1) + fib(n-2)
```

Q22 Generate Prime Numbers Between Two Ranges

Objective

Display all prime numbers between a given range [start, end].

Concept	Sieve concept: test each number in range for primality. Reuse the is_prime function (modularity). Collect primes in a list.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input start and end
Step 3: For each number in range
Step 4: Check if prime
Step 5: If yes, add to list and print
Step 6: Display count
Step 7: Stop

Flow Explanation

Read range → Check each number → Filter primes → Display all primes and total count.

Python Code

```
# Program 22: Primes Between Two Ranges

import math

def is_prime(num):
    if num < 2: return False
    for i in range(2, int(math.sqrt(num)) + 1):
        if num % i == 0: return False
    return True

start = int(input('Enter start of range: '))
end   = int(input('Enter end of range: '))

primes = [n for n in range(start, end+1) if is_prime(n)]  # List comprehension

print(f'\nPrime numbers between {start} and {end}:')
print(primes)
print(f'Total prime numbers found: {len(primes)}')
```

Sample Input	Sample Output
Start: 10 End: 50	Prime numbers between 10 and 50: [11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47] Total prime numbers found: 11

📖 Code Explanation (Line by Line)

Lines 5-9: Reusable is_prime() function (defined once, called many times)

Line 14: List comprehension — elegant one-line filter creating a list of primes

Line 17: len(primes) gives count of primes found

⏱ Time Complexity	🌍 Real-World Use Case
O((end-start) × √end) — O(N√N) for range of size N.	Cryptographic key generation, mathematical research, generating prime lookup tables.

🎓 Viva Questions & Answers

Q: What is a list comprehension?

A: A concise way to create lists: [expression for item in iterable if condition]. Equivalent to a for loop with append.

Q: What is the Sieve of Eratosthenes?

A: An ancient algorithm that eliminates all multiples of each prime starting from 2 to efficiently find all primes up to N.

Q: Are there infinitely many primes?

A: Yes. Euclid proved this around 300 BC. There is no largest prime number.

Q23 Pattern Printing — Right Triangle

Objective

Print a right-angled triangle pattern using asterisks (*) and nested loops.

Concept	Nested loops: outer loop controls rows, inner loop controls columns/characters. Number of stars in row i equals i. String repetition simplifies inner loop.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input N (number of rows)
Step 3: For each row i from 1 to N
Step 4: Print i stars
Step 5: Move to next line
Step 6: Stop

Flow Explanation

Outer loop → row number i → inner loop prints i stars → newline → repeat for N rows.

Python Code

```
# Program 23: Right Triangle Pattern

n = int(input('Enter number of rows: '))

print('\nRight Triangle Pattern:')
for i in range(1, n + 1):   # Outer loop: controls rows
    for j in range(1, i + 1): # Inner loop: prints i stars in row i
        print('*', end='')
    print()                  # Move to next line after each row
```

Sample Input	Sample Output
Enter number of rows: 5	* ** *** **** *****

Code Explanation (Line by Line)

Line 6: Outer loop — i goes 1,2,3,4,5
Line 7: Inner loop — j goes from 1 to i (so row 3 prints 3 stars)
Line 8: print('*', end='') prevents newline between stars
Line 9: print() adds newline after each row is complete

Time Complexity	Real-World Use Case
$O(N^2)$ — total stars = 1+2+...+N = N(N+1)/2.	Educational pattern generation, ASCII art, understanding nested loop concepts.

Viva Questions & Answers

Q: Why do we need nested loops for patterns?

A: Patterns are 2D structures — rows and columns. The outer loop handles rows, inner loop handles characters within each row.

Q: How would you print an inverted triangle?

A: Use range(n, 0, -1) in the outer loop to start with N stars and decrease.

Q: What does end='' do in print()?

A: Overrides the default end='\n', so print() doesn't add a newline. Multiple print() calls then output on the same line.

Alternative Method

```
# Using string repetition (simpler)
for i in range(1, n+1):     print('*' * i)
```

Q24 Pattern Printing — Pyramid

Objective

Print a centered pyramid pattern using spaces and asterisks.

Concept	Pyramid has two components: leading spaces (centering) and odd number of stars. Row i has (n-i) spaces and (2i-1) stars. Both decrease/increase by pattern.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input N rows
Step 3: For row i from 1 to N
Step 4: Print (N-i) spaces
Step 5: Print (2i-1) asterisks
Step 6: Newline
Step 7: Stop

Flow Explanation

Calculate spaces and stars for each row → Print spaces first → Print odd number of stars → Newline → Next row.

Python Code

```
# Program 24: Pyramid Pattern

n = int(input('Enter number of rows: '))

print('\nPyramid Pattern:')
for i in range(1, n + 1):
    spaces = n - i          # Leading spaces decrease each row
    stars  = 2 * i - 1      # Stars: 1, 3, 5, 7... (odd numbers)
    print(' ' * spaces + '*' * stars)  # Combine spaces and stars
```

Sample Input	Sample Output
Enter number of rows: 5	* *** ***** ******* *********

Code Explanation (Line by Line)

Line 7: spaces = n-i → Row 1: 4 spaces, Row 2: 3, ..., Row 5: 0

Line 8: stars = 2i-1 → Row 1: 1, Row 2: 3, ..., Row 5: 9

Line 9: String repetition creates space prefix + star pattern

Time Complexity	Real-World Use Case
O(N²) — total characters proportional to N².	ASCII art, console UI decoration, understanding mathematical patterns in loops.

Viva Questions & Answers

Q: Why are there always odd numbers of stars in a pyramid?

A: Row i has 2i-1 stars. The formula 2i-1 always produces odd numbers: 1,3,5,7,9...

Q: How to print a hollow pyramid?

A: Print '*' only at the first position, last position, and the top row of each row; fill middle with spaces.

Q: What is the formula for total characters in an N-row pyramid?

A: Total stars = N^2 (sum of 2i-1 for i=1..N = N^2). Total spaces = N(N-1)/2.

Q25 Pattern Printing — Diamond

Objective

Print a diamond shape by combining an upper pyramid and an inverted lower pyramid.

Concept	Diamond = upper pyramid (1 to N rows) + lower inverted pyramid (N-1 to 1). Reuse pyramid logic for both halves with loop direction reversed for second half.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input N

Step 3: Upper half: for i from 1 to N → print (N-i) spaces + (2i-1) stars

Step 4: Lower half: for i from N-1 to 1 → print (N-i) spaces + (2i-1) stars

Step 5: Stop

Flow Explanation

Build top half (expanding) → Peak row → Build bottom half (contracting) → Diamond complete.

Python Code

```
# Program 25: Diamond Pattern

n = int(input('Enter number of rows (half-size): '))

print('\nDiamond Pattern:')

# Upper half (including middle row)
for i in range(1, n + 1):
    print(' ' * (n - i) + '*' * (2 * i - 1))

# Lower half (mirror of upper, without middle)
for i in range(n - 1, 0, -1):   # range goes backwards!
    print(' ' * (n - i) + '*' * (2 * i - 1))
```

Sample Input	Sample Output
Enter number of rows: 4	* *** ***** ******* ***** *** *

Code Explanation (Line by Line)

Lines 8-9: Upper half — same as pyramid pattern

Lines 12-13: range(n-1, 0, -1) counts DOWN: n-1, n-2, ..., 1

Same formula works for lower half — decreasing i reduces stars and increases spaces

Time Complexity	Real-World Use Case

$O(N^2)$ — two half-pyramids each $O(N^2/2)$.	Decorative console output, understanding bidirectional loops, geometric pattern generation.

🎓 Viva Questions & Answers

Q: How many total rows does a diamond with n=4 have?

A: 2n-1 = 7 rows (4 upper including middle + 3 lower).

Q: What does range(n-1, 0, -1) produce for n=4?

A: 3, 2, 1 — starts at n-1=3, stops before 0, step -1.

Q: How would you print a hollow diamond?

A: Print '*' only at the edges of each row, filling the middle with spaces for interior rows.

SECTION 4: STRING PROGRAMS

Questions 26-30: String Manipulation

Q26 Reverse a String

Objective

Reverse a given string and display the result.

Concept	Python strings are sequences. Slice notation [start:stop:step] accesses substrings. [::-1] reverses by stepping -1 from end to start.

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input a string

Step 3: Use slice [::-1] to reverse

Step 4: Display reversed string

Step 5: Stop

Flow Explanation

Read string → Apply reverse slice → Display reversed version.

Python Code

```
# Program 26: Reverse a String

s = input('Enter a string: ')

# Method: Python slice with step -1
reversed_s = s[::-1]   # [start:stop:step] — step=-1 reverses

print(f'Original : {s}')
print(f'Reversed : {reversed_s}')
```

Sample Input	Sample Output
Enter a string: Hello Python	Original : Hello Python Reversed : nohtyP olleH

Code Explanation (Line by Line)

Line 3: input() reads entire line including spaces

Line 6: s[::-1] — start and stop defaults give full string; step=-1 reverses direction

Strings are immutable — reversal creates a new string

Time Complexity	Real-World Use Case

O(N) — must copy N characters.	Palindrome verification, data encryption, reversing display text in UI.

Viva Questions & Answers

Q: What is string slicing?

A: Slicing extracts a substring: s[start:stop:step]. s[1:4] gives characters at index 1,2,3. Defaults: start=0, stop=len, step=1.

Q: What does s[::-1] mean?

A: Starts from the end (default), goes to the beginning (default), with step -1 (backwards) — effectively reversing the string.

Q: Are Python strings mutable?

A: No. Strings are immutable. Operations like reverse/replace create new strings rather than modifying in place.

Alternative Method

```
# Using reversed() and join
print(''.join(reversed(s)))

# Using a loop
rev = '' for ch in s:     rev = ch + rev print(rev)
```

Q27 Check Palindrome String

Objective

Determine whether a given string reads the same forwards and backwards (case-insensitive).

Concept	Palindrome strings read identically from both ends. Convert to lowercase for case-insensitive check. Remove spaces for better comparison.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input string
Step 3: Convert to lowercase, remove spaces
Step 4: Reverse cleaned string
Step 5: Compare original cleaned with reversed
Step 6: Print result
Step 7: Stop

Flow Explanation

Read → Normalize (lower+no spaces) → Reverse → Compare → Print verdict.

Python Code

```
# Program 27: Palindrome String Check

s = input('Enter a string: ')

# Normalize: lowercase and remove spaces
cleaned = s.lower().replace(' ', '')  # Method chaining

# Reverse and compare
if cleaned == cleaned[::-1]:
    print(f'"{s}" is a Palindrome string ✓')
else:
    print(f'"{s}" is NOT a Palindrome string ✗')
```

Sample Input	Sample Output
Enter a string: racecar	"racecar" is a Palindrome string ✓

Code Explanation (Line by Line)

Line 6: .lower() converts to lowercase; .replace(' ','') removes spaces; chained method calls

Line 9: Compare cleaned string with its reverse

'A man a plan a canal Panama' → 'amanaplanacanalpanama' → palindrome ✓

Time Complexity	Real-World Use Case
O(N) — string reversal and comparison.	Word game validation, DNA sequence analysis, text processing.

Viva Questions & Answers

Q: Give 5 palindrome words/sentences.

A: 'racecar', 'level', 'madam', 'A man a plan a canal Panama', 'Was it a car or a cat I saw'.

Q: What is method chaining?

A: Calling multiple methods sequentially: s.lower().replace(' ','') — result of lower() becomes the object for replace().

Q: How to ignore punctuation in palindrome check?

A: Use re.sub(r'[^a-z0-9]','',s.lower()) to keep only alphanumeric characters.

Q28 Count Vowels and Consonants

Objective

Count the number of vowels and consonants in a given string.

Concept	Iterate through each character. Check membership in a vowel set using 'in'. Skip non-alphabetic characters with isalpha(). String methods process text efficiently.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input string
Step 3: Initialize vowel_count=0, consonant_count=0
Step 4: For each character
Step 5: If alpha and vowel: vowel_count++
Step 6: Elif alpha: consonant_count++
Step 7: Print counts
Step 8: Stop

Flow Explanation

Iterate chars → Check if letter → If vowel: count vowel → Else: count consonant → Display totals.

Python Code

```
# Program 28: Count Vowels and Consonants

s = input('Enter a string: ')

vowels = 'aeiouAEIOU'   # String of all vowels
vowel_count = 0
consonant_count = 0

for char in s:
    if char.isalpha():           # Check if character is a letter
        if char in vowels:       # Check if it is a vowel
            vowel_count += 1
        else:
            consonant_count += 1  # All non-vowel letters are consonants

print(f'String     : {s}')
print(f'Vowels     : {vowel_count}')
print(f'Consonants : {consonant_count}')
```

Sample Input	Sample Output
Enter a string: Hello World	String : Hello World Vowels : 3 Consonants : 7

Code Explanation (Line by Line)

Line 5: Define vowel membership set as a string

Line 10: isalpha() returns True only for letters (skips spaces, digits, punctuation)

Line 11: 'in' operator checks if char exists in the vowels string — O(1) for small sets

'Hello World': H,l,l = 3 consonants; e,o,o = 3 vowels; W,r,l,d = 4 consonants

Time Complexity	Real-World Use Case
O(N) — one pass through the string.	Text analysis, language processing, readability score calculation, spelling tools.

Viva Questions & Answers

Q: What does isalpha() return?

A: Returns True if all characters in the string are alphabetic letters (A-Z, a-z). Returns False for digits, spaces, punctuation.

Q: What is the 'in' operator for strings?

A: 'in' tests membership — 'a' in 'aeiou' returns True. For strings, it checks if the left operand is a substring of the right.

Q: How many vowels and consonants are in 'Python'?

A: Vowels: o (1). Consonants: P,y,t,h,n (5). Total letters: 6.

Alternative Method

```
# Using count() method
vowels='aeiouAEIOU' vc = sum(s.count(v) for v in vowels) cc = sum(1 for c in s if c.isalpha() and c not in vowels) print(f'Vowels:{vc}, Consonants:{cc}')
```

Q29 Count Words in a Sentence

Objective

Count the total number of words in an input sentence.

Concept	split() splits a string by whitespace and returns a list of words. len() gives list length. strip() removes leading/trailing whitespace before processing.

Algorithm (Step-by-Step)

Step 1: Start
Step 2: Input sentence
Step 3: Strip whitespace
Step 4: Split by spaces into word list
Step 5: Count elements in list
Step 6: Display count
Step 7: Stop

Flow Explanation

Read sentence → Strip → Split → Count list elements → Display.

Python Code

```
# Program 29: Count Words in a Sentence

sentence = input('Enter a sentence: ')

# strip() removes leading/trailing spaces
# split() splits on whitespace and returns a list of words
words = sentence.strip().split()

word_count = len(words)  # Count elements in the list

print(f'Sentence   : "{sentence}"')
print(f'Word count: {word_count}')
print(f'Words      : {words}')
```

Sample Input	Sample Output
Enter a sentence: The quick brown fox jumps over the lazy dog	Word count: 9 Words : ['The', 'quick', 'brown', 'fox', 'jumps', 'over', 'the', 'lazy', 'dog']

Code Explanation (Line by Line)

Line 7: strip() handles extra spaces at start/end; split() handles multiple spaces between words

Line 9: len(words) gives number of words

split() with no arguments splits on any whitespace (spaces, tabs, newlines) and ignores extras

Time Complexity	Real-World Use Case
O(N) — scan entire string once.	Word processors (word count feature), search engines (indexing), social media character limits.

Viva Questions & Answers

Q: What does split() return?

A: A list of strings. 'hello world'.split() returns ['hello', 'world']. Without arguments, splits on any whitespace.

Q: What does strip() do?

A: Removes leading and trailing whitespace (spaces, tabs, newlines). 'hello '.strip() returns 'hello'.

Q: How to count words matching a specific word?

A: words.count('the') — returns how many times 'the' appears in the word list.

Alternative Method

```
# Using len and split directly
n = len(input('Sentence: ').split()) print(f'Words: {n}')

# Count specific word occurrences
print(sentence.lower().split().count('the'))
```

Q30 Find Frequency of Characters

Objective

Count how many times each character appears in a given string.

Concept	Dictionary (dict) maps characters to their counts. get() provides default value. Counter from collections automates frequency counting. Dictionary traversal with items().

Algorithm (Step-by-Step)

Step 1: Start

Step 2: Input string

Step 3: Create empty dictionary freq

Step 4: For each character in string: freq[char] = freq.get(char,0) + 1

Step 5: Print all character-frequency pairs

Step 6: Stop

Flow Explanation

Iterate string → For each char → Look up in dict (default 0) → Increment → Store → Display sorted results.

Python Code

```
# Program 30: Character Frequency

s = input('Enter a string: ')

# Build frequency dictionary
freq = {}   # Empty dictionary
for char in s:
    freq[char] = freq.get(char, 0) + 1  # get() returns 0 if key not found

# Display results sorted by character
print(f'\nCharacter frequencies in "{s}":')
print('-' * 30)
for char, count in sorted(freq.items()):
    print(f"  '{char}' : {count} time(s)")
```

Sample Input	Sample Output
Enter a string: hello	Character frequencies in "hello": 'e' : 1 time(s) 'h' : 1 time(s) 'l' : 2 time(s) 'o' : 1 time(s)

Code Explanation (Line by Line)

Line 6: {} creates an empty dictionary

Line 8: freq.get(char, 0) returns existing count or 0 if first occurrence; then +1

Line 13: freq.items() returns (key,value) pairs; sorted() alphabetizes by character

Line 14: Unpack each tuple: char gets key, count gets value

Time Complexity	Real-World Use Case
O(N) — single pass; dictionary operations are O(1) average.	Text analytics, cipher analysis, data compression (Huffman encoding), spam detection.

Viva Questions & Answers

Q: What is a dictionary in Python?

A: A dictionary is an unordered collection of key-value pairs. Keys must be unique and hashable. Values can be any type. Syntax: {key: value}.

Q: What does dict.get(key, default) do?

A: Returns the value for key if it exists; returns default (here 0) if key is not found — avoids KeyError.

Q: How to find the most frequent character?

A: max(freq, key=freq.get) returns the key with maximum value. Or use Counter from collections: Counter(s).most_common(1).

Q: What does .items() return?

A: Returns a view of (key, value) tuple pairs from the dictionary, which can be iterated.

Alternative Method

```
# Using Counter (one-liner)
from collections import Counter freq = Counter(s) print(freq.most_common())
```

MINI PROJECTS

Apply your Python knowledge in these complete project applications.

P1 Student Result Management System

Project Objective

A console application to enter student marks, calculate percentages, assign grades, and store records.

Features

- Enter marks for multiple subjects
- Automatic percentage and grade calculation
- View all stored records
- Find the topper
- Store data in a list of dictionaries

Complete Python Code

```
# Mini Project 1: Student Result Management System

students = []   # List to store all student records

def get_grade(percentage):
    if percentage >= 90: return 'A+'
    elif percentage >= 75: return 'A'
    elif percentage >= 60: return 'B'
    elif percentage >= 50: return 'C'
    elif percentage >= 40: return 'D'
    else: return 'F'

def add_student():
    name = input('Enter student name: ')
    marks = []
    subjects = ['Math', 'Science', 'English', 'History', 'Computer']
    for sub in subjects:
        m = float(input(f'  Marks in {sub} (out of 100): '))
        marks.append(m)
    total = sum(marks)
    percentage = (total / 500) * 100
    grade = get_grade(percentage)
    students.append({'name':name,'marks':marks,'total':total,
                     'percentage':percentage,'grade':grade})
    print(f'\n✓ Record added for {name}')

def show_all():
    if not students:
        print('No records found!'); return
    print(f'\n{"Name":<15}{"Total":<10}{"Percentage":<12}{"Grade"}')
    print('-' * 45)
    for s in students:
```

```
    print(f'{s["name"]:<15}{s["total"]:<10.1f}{s["percentage"]:<12.2f}{s["grade"]}')

def find_topper():
    if not students: print('No records!'); return
    topper = max(students, key=lambda s: s['percentage'])
    print(f'\nTopper: {topper["name"]} with {topper["percentage"]:.2f}%')

# Main menu loop
while True:
    print('\n--- Student Result System ---')
    print('1. Add Student  2. View All  3. Find Topper  4. Exit')
    choice = input('Choice: ')
    if choice=='1': add_student()
    elif choice=='2': show_all()
    elif choice=='3': find_topper()
    elif choice=='4': print('Goodbye!'); break
    else: print('Invalid choice!')
```

📖 Explanation

students = [] — A list of dictionaries. Each dictionary stores one student's complete record.
get_grade() — Reusable function that maps percentage to grade. Defined once, called multiple times.
add_student() — Loops through 5 subjects, collects marks, computes total and percentage, stores in students list.
show_all() — Formatted table output using f-string alignment specifiers (<15 left-aligns in 15 chars).
find_topper() — Uses max() with a lambda function as the key (sorts by percentage, returns student dict).
Main while loop — Menu-driven program that keeps running until user chooses to exit.

🚀 Future Enhancements

→ Save/load data to/from files using pickle or JSON
→ Add search by name functionality
→ Export results to CSV using csv module
→ Add input validation (marks must be 0-100)
→ GUI version using tkinter

🎓 Viva Questions

Q: What is a list of dictionaries?

A: A list where each element is a dictionary. Commonly used to represent records (like rows in a table). Access: students[0]['name'].

Q: What is a lambda function?

A: An anonymous one-line function. lambda x: x['percentage'] creates a function that takes x and returns x['percentage']. Used as keys in sort/max.

Q: How does max() with key= work?

A: max(students, key=lambda s: s['percentage']) applies the function to each element and returns the element with the maximum result.

P2 ATM Simulation System

🎯 Project Objective

Simulate basic ATM operations: check balance, deposit, withdraw with PIN authentication.

✨ Features

- PIN authentication with 3 attempts
- Check account balance
- Deposit money
- Withdraw with balance validation
- Transaction history
- Mini statement

💻 Complete Python Code

```
# Mini Project 2: ATM Simulation System

# Account data
account = {
    'name': 'Rahul Sharma',
    'account_no': '1234567890',
    'pin': '1234',
    'balance': 10000.00,
    'transactions': []
}

def authenticate():
    for attempt in range(3):          # 3 attempts allowed
        pin = input('Enter PIN: ')
        if pin == account['pin']:
            return True
        remaining = 2 - attempt
        if remaining > 0:
            print(f'Wrong PIN! {remaining} attempts left.')
    print('Account locked! Too many wrong attempts.')
    return False

def check_balance():
    print(f'\nAccount: {account["account_no"]}')
    print(f'Name   : {account["name"]}')
    print(f'Balance: ₹{account["balance"]:,.2f}')

def deposit():
    amount = float(input('Enter deposit amount: ₹'))
    if amount <= 0:
        print('Invalid amount!'); return
    account['balance'] += amount
    account['transactions'].append(f'Deposited ₹{amount:,.2f}')
    print(f'✓ ₹{amount:,.2f} deposited. New balance: ₹{account["balance"]:,.2f}')

def withdraw():
```

```
    amount = float(input('Enter withdrawal amount: ₹'))
    if amount <= 0: print('Invalid amount!'); return
    if amount > account['balance']:
        print('Insufficient balance!'); return
    account['balance'] -= amount
    account['transactions'].append(f'Withdrew  ₹{amount:,.2f}')
    print(f'✓ ₹{amount:,.2f} dispensed. Remaining: ₹{account["balance"]:,.2f}')

def mini_statement():
    print(f'\n--- Mini Statement ---')
    if not account['transactions']:
        print('No transactions yet.'); return
    for i, t in enumerate(account['transactions'][-5:], 1):  # Last 5
        print(f'  {i}. {t}')
    print(f'Current Balance: ₹{account["balance"]:,.2f}')

# Main ATM
print('=== WELCOME TO PYTHON BANK ATM ===')
if authenticate():
    print(f'\nWelcome, {account["name"]}!')
    while True:
        print('\n1.Balance  2.Deposit  3.Withdraw  4.Statement  5.Exit')
        ch = input('Select: ')
        if ch=='1': check_balance()
        elif ch=='2': deposit()
        elif ch=='3': withdraw()
        elif ch=='4': mini_statement()
        elif ch=='5': print('Thank you! Goodbye.'); break
        else: print('Invalid option!')
```

🕮 Explanation

account dictionary — Stores all account data (name, PIN, balance, transaction history) in one structure.
authenticate() — 3 attempts loop; returns True on correct PIN, False and locks after 3 failures.
deposit()/withdraw() — Validate amount, update balance, record transaction string in history list.
:,.2f format — Adds comma separators to large numbers: 10000 → 10,000.00.
enumerate(list[-5:], 1) — Gets last 5 transactions and numbers them from 1.
Nested function approach — Keeps code modular and each operation in its own function.

🚀 Future Enhancements

→ Multiple account support with file storage
→ Transfer between accounts
→ OTP-based authentication
→ Receipt printing to file
→ Daily withdrawal limits

🎓 Viva Questions

Q: How does the 3-attempt lock work?

A: A for loop runs 3 times. Each failed attempt decrements remaining counter. If all 3 fail, function returns False and ATM is locked.

Q: What does :,.2f mean?

A: : starts format spec; , adds comma thousands separator; .2f means 2 decimal places fixed. 10000 → '10,000.00'.

Q: What is the difference between a list and a dictionary?

A: Lists are ordered sequences accessed by index [0,1,2...]. Dictionaries are key-value mappings accessed by named keys like ['name'].

P3 Number Guessing Game

🎯 Project Objective

A fun game where the computer picks a random number and the player tries to guess it with hints.

✨ Features

- Random number generation
- Hot/Cold hints (higher/lower)
- Attempt counter
- Multiple rounds
- Best score tracking

💻 Complete Python Code

```
# Mini Project 3: Number Guessing Game

import random

def play_round(low=1, high=100):
    secret = random.randint(low, high)  # Random number inclusive
    attempts = 0
    print(f'\nGuess a number between {low} and {high}!')

    while True:
        try:
            guess = int(input('Your guess: '))
        except ValueError:
            print('Please enter a valid number!'); continue

        attempts += 1

        if guess == secret:
            print(f'🎉 Correct! You got it in {attempts} attempt(s)!')
            return attempts
        elif guess < secret:
            print('📈 Too LOW! Go higher.')
        else:
            print('📉 Too HIGH! Go lower.')

# Main game
print('=== NUMBER GUESSING GAME ===')
best_score = float('inf')   # Infinity as initial best
rounds = 0

while True:
    attempts = play_round()
    rounds += 1
    if attempts < best_score:
        best_score = attempts
        print(f'⏳ New best score: {best_score} attempts!')
```

```
    print(f'\nRound {rounds} complete. Best: {best_score} attempts.')
    again = input('Play again? (y/n): ').lower()
    if again != 'y':
        print(f'\nGame Over! Played {rounds} rounds. Best score: {best_score}')
        break
```

🕮 Explanation

random.randint(low, high) — Generates random integer inclusive on both ends (e.g., randint(1,100) can return 1 or 100).

try/except ValueError — Handles invalid input (letters, decimals) gracefully without crashing.

float('inf') — Represents positive infinity. Any real number will be less, making it ideal as an initial minimum.

return attempts — Function returns the number of attempts for score tracking.

Best score tracking — Compares each round's attempts with the running minimum.

🚀 Future Enhancements

→ Difficulty levels (Easy: 1-50, Hard: 1-500)

→ Time limit per guess

→ Hint system after 5 wrong guesses

→ Leaderboard with player names

→ Graphical version with tkinter

🎓 Viva Questions

Q: What does random.randint(1,100) do?

A: Returns a random integer N such that 1 <= N <= 100 (inclusive on both ends). Uses the Mersenne Twister algorithm.

Q: What is float('inf')?

A: Positive infinity in Python. Any finite number is less than inf, making it useful as an initial value for finding minimums.

Q: What is try/except used for?

A: Exception handling — try block runs code that might fail; except catches specific errors (like ValueError) and handles them gracefully.

P4 To-Do List Manager

🎯 Project Objective

A task management application to add, view, complete, and delete tasks from a to-do list.

✨ Features

- Add new tasks
- Mark tasks as complete
- Delete tasks
- View pending and completed tasks
- Task priority levels

💻 Complete Python Code

```
# Mini Project 4: To-Do List Manager

tasks = []  # List of task dictionaries
task_id = 1

def add_task():
    global task_id
    title = input('Task title: ')
    priority = input('Priority (High/Medium/Low): ').capitalize()
    tasks.append({'id':task_id,'title':title,'priority':priority,'done':False})
    print(f'✓ Task #{task_id} added!')
    task_id += 1

def view_tasks(show_done=False):
    filtered = [t for t in tasks if t['done'] == show_done]
    status = 'Completed' if show_done else 'Pending'
    print(f'\n--- {status} Tasks ---')
    if not filtered:
        print(f'No {status.lower()} tasks.'); return
    for t in filtered:
        icon = '✓' if t['done'] else 'o'
        print(f'  {icon} [{t["id"]}] {t["title"]} ({t["priority"]})')

def complete_task():
    view_tasks(show_done=False)
    try:
        tid = int(input('Enter task ID to mark complete: '))
        for t in tasks:
            if t['id'] == tid:
                t['done'] = True
                print(f'✓ Task #{tid} marked complete!')
                return
        print('Task not found!')
    except ValueError: print('Invalid ID!')

def delete_task():
    try:
```

```
        tid = int(input('Enter task ID to delete: '))
        for i, t in enumerate(tasks):
            if t['id'] == tid:
                tasks.pop(i)
                print(f'✓ Task #{tid} deleted!')
                return
        print('Task not found!')
    except ValueError: print('Invalid ID!')

print('=== TO-DO LIST MANAGER ===')
while True:
    print('\n1.Add  2.Pending  3.Done  4.Complete  5.Delete  6.Exit')
    ch = input('Choice: ')
    if ch=='1': add_task()
    elif ch=='2': view_tasks(show_done=False)
    elif ch=='3': view_tasks(show_done=True)
    elif ch=='4': complete_task()
    elif ch=='5': delete_task()
    elif ch=='6': print('Goodbye!'); break
    else: print('Invalid!')
```

🕮 Explanation

tasks = [] — List of dictionaries. Each task has id, title, priority, and done (boolean) fields.

global task_id — Needed to modify the global variable from inside a function.

List comprehension filter — [t for t in tasks if t['done'] == show_done] creates filtered list.

tasks.pop(i) — Removes element at index i from the list.

Parameterized view_tasks() — show_done parameter makes one function serve two purposes (pending/completed).

🚀 Future Enhancements

→ Save tasks to JSON file for persistence

→ Due dates and reminders

→ Categories/labels for tasks

→ Priority-based sorting

→ GUI using tkinter or web interface using Flask

🎓 Viva Questions

Q: What is the global keyword?

A: global declares that a variable inside a function refers to the global (module-level) variable, allowing modification. Without it, Python creates a local variable.

Q: What does list.pop(i) do?

A: Removes and returns the element at index i. list.pop() without argument removes the last element.

Q: How do you search a list of dictionaries by a key value?

A: Use a loop: for t in tasks: if t['id'] == target: ... Or: next((t for t in tasks if t['id']==target), None).

P5 Password Generator

🎯 Project Objective

Generate strong random passwords based on user-specified length and character type preferences.

✨ Features

- Customizable length
- Include/exclude uppercase, lowercase, digits, symbols
- Password strength rating
- Copy-ready output
- Generate multiple passwords

💻 Complete Python Code

```
# Mini Project 5: Password Generator

import random
import string

def generate_password(length=12, use_upper=True, use_lower=True,
                      use_digits=True, use_symbols=True):
    charset = ''
    if use_upper:   charset += string.ascii_uppercase  # A-Z
    if use_lower:   charset += string.ascii_lowercase  # a-z
    if use_digits:  charset += string.digits           # 0-9
    if use_symbols: charset += string.punctuation      # !@#$...

    if not charset:
        return 'Error: Select at least one character type!'

    # Ensure at least one char from each required type
    password = []
    if use_upper:   password.append(random.choice(string.ascii_uppercase))
    if use_lower:   password.append(random.choice(string.ascii_lowercase))
    if use_digits:  password.append(random.choice(string.digits))
    if use_symbols: password.append(random.choice(string.punctuation))

    # Fill remaining length
    remaining = length - len(password)
    password += random.choices(charset, k=remaining)

    # Shuffle to avoid predictable pattern
    random.shuffle(password)
    return ''.join(password)

def get_strength(length, types_used):
    score = length // 4 + types_used * 2
    if score >= 10: return '🔓 Very Strong'
    elif score >= 7: return '🔑 Strong'
    elif score >= 4: return '⚠️  Moderate'
```

```
    else: return '❌ Weak'

print('=== PASSWORD GENERATOR ===')
length = int(input('Password length (8-32): '))
length = max(8, min(32, length))  # Clamp between 8-32

upper   = input('Uppercase (Y/n): ').lower() != 'n'
lower   = input('Lowercase (Y/n): ').lower() != 'n'
digits  = input('Digits    (Y/n): ').lower() != 'n'
symbols = input('Symbols   (Y/n): ').lower() != 'n'

types = sum([upper, lower, digits, symbols])

print('\n--- Generated Passwords ---')
for i in range(5):   # Generate 5 options
    pwd = generate_password(length, upper, lower, digits, symbols)
    strength = get_strength(length, types)
    print(f'  {i+1}. {pwd:<35} {strength}')
```

🕮 Explanation

string module — Provides ready-made character sets: ascii_uppercase='ABCDE...Z', digits='0123456789', punctuation='!@#...'.

Guaranteed inclusion — Append one character from each selected type before random fill (ensures policy compliance).

random.choices(charset, k=n) — Picks n random items from charset (with replacement).

random.shuffle(password) — Randomizes the list order to remove positional bias from the guaranteed chars.

''.join(password) — Converts list of characters to a string.

🚀 Future Enhancements

→ Save passwords securely to encrypted file

→ Password strength meter bar

→ Avoid ambiguous characters (0,O,l,1)

→ Pronounceable password option

→ pyperclip integration for clipboard copy

🎓 Viva Questions

Q: What is the string module?

A: A built-in Python module containing useful string constants: string.ascii_letters, string.digits, string.punctuation, string.whitespace, etc.

Q: What does random.shuffle() do?

A: Shuffles a list in-place (modifies the original). Returns None. Uses Fisher-Yates algorithm.

Q: Why is random.choices() used instead of random.choice()?

A: random.choice() picks ONE item. random.choices(population, k=n) picks n items WITH replacement. random.sample() picks n WITHOUT replacement.

BONUS SECTION

Part A: Top 50 Python Viva Questions with Answers

Q: What is Python?

A: Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum in 1991. It emphasizes code readability and simplicity.

Q: What are Python's key features?

A: Easy to learn, interpreted, dynamically typed, object-oriented, large standard library, platform-independent, open source.

Q: What is the difference between compiled and interpreted languages?

A: Compiled languages (C, C++) translate entire code to machine code before execution. Interpreted languages (Python) execute code line-by-line via an interpreter.

Q: What is a variable in Python?

A: A variable is a named memory location that stores a value. Python is dynamically typed — you don't declare types. Example: x = 10.

Q: What are Python's built-in data types?

A: int, float, complex, str, list, tuple, dict, set, frozenset, bool, bytes, NoneType.

Q: What is the difference between list and tuple?

A: Lists are mutable (can be changed). Tuples are immutable (cannot be changed). Lists use [], tuples use (). Tuples are faster.

Q: What is a dictionary in Python?

A: An unordered (Python 3.7+ ordered) collection of key-value pairs. Keys must be unique and immutable. Example: {'name': 'Alice', 'age': 20}.

Q: What is the difference between == and is?

A: == compares values (equality). 'is' compares identity (same object in memory). Example: [1,2] == [1,2] is True, but [1,2] is [1,2] is False.

Q: What is None in Python?

A: None is Python's null value — represents 'no value' or 'nothing'. It is the sole instance of NoneType. Check with: if x is None.

Q: What are Python's comparison operators?

A: == (equal), != (not equal), > (greater), < (less), >= (greater or equal), <= (less or equal). All return True or False.

Q: What is indentation in Python?

A: Python uses whitespace indentation (typically 4 spaces) to define code blocks instead of curly braces {}. Incorrect indentation causes IndentationError.

Q: What is the difference between break and continue?

A: break exits the loop entirely. continue skips the rest of the current iteration and goes to the next iteration.

Q: What is a function in Python?

A: A reusable block of code defined with def keyword. Functions reduce code repetition and improve organization. Example: def greet(name): return 'Hello ' + name.

Q: What are *args and **kwargs?

A: *args accepts variable number of positional arguments as a tuple. **kwargs accepts variable keyword arguments as a dictionary.

Q: What is a lambda function?

A: An anonymous, one-line function. Syntax: lambda parameters: expression. Example: square = lambda x: x**2. Used in map(), filter(), sorted() key.

Q: What is list comprehension?

A: A concise way to create lists: [expression for item in iterable if condition]. Example: squares = [x**2 for x in range(10)].

Q: What is a module in Python?

A: A file containing Python code (functions, classes, variables). Import with 'import module_name'. Examples: math, random, os, sys.

Q: What is the difference between import and from...import?

A: import math — access with math.sqrt(). from math import sqrt — access directly as sqrt(). from math import * — imports all (not recommended).

Q: What are Python's logical operators?

A: and — True if both True; or — True if at least one True; not — reverses Boolean value.

Q: What is type casting in Python?

A: Converting one data type to another. int('5') → 5, float(3) → 3.0, str(42) → '42', list((1,2)) → [1,2].

Q: What is the difference between append() and extend()?

A: append() adds a single element to list end. extend() adds all elements of an iterable. [1,2].append([3,4]) → [1,2,[3,4]]; extend → [1,2,3,4].

Q: What is slicing?

A: Extracting a portion of a sequence. Syntax: seq[start:stop:step]. s[1:4] returns index 1,2,3. s[::-1] reverses.

Q: What is the range() function?

A: Generates an immutable sequence of numbers. range(stop), range(start,stop), range(start,stop,step). Example: range(1,10,2) → 1,3,5,7,9.

Q: What is the difference between local and global variables?

A: Local variables exist only inside a function. Global variables exist at module level. Use 'global' keyword to modify global variable inside a function.

Q: What is recursion?

A: A function calling itself. Must have a base case to stop. Example: factorial(n) = n * factorial(n-1) with base case factorial(0)=1.

Q: What is exception handling?

A: Handling runtime errors gracefully using try-except-else-finally blocks. Prevents program crash. Example: try: x=1/0 except ZeroDivisionError: print('Error').

Q: What is OOP?

A: Object-Oriented Programming — organizing code around objects with attributes (data) and methods (functions). Python's 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction.

Q: What is a class and an object?

A: A class is a blueprint/template. An object is an instance created from a class. class Car: pass; my_car = Car() — my_car is an object.

Q: What is __init__() in Python?

A: The constructor method called automatically when an object is created. Initializes object attributes. 'self' refers to the current object instance.

Q: What is inheritance?

A: A class can inherit attributes and methods from a parent class. Promotes code reuse. Syntax: class Child(Parent):. Python supports multiple inheritance.

Q: What is the difference between a list and a set?

A: Lists maintain order and allow duplicates. Sets are unordered, contain only unique elements, and support mathematical set operations (union, intersection).

Q: What is a file operation in Python?

A: open(filename, mode) opens a file. Modes: 'r' (read), 'w' (write), 'a' (append), 'rb' (binary). Use 'with' statement for automatic closing.

Q: What is the 'with' statement?

A: Context manager that ensures resources are properly released. with open('file.txt') as f: automatically closes the file even if an error occurs.

Q: What is PEP 8?

A: Python Enhancement Proposal 8 — the official Python style guide. Covers naming conventions, indentation, line length, comments, and code layout.

Q: What is the difference between deep copy and shallow copy?

A: Shallow copy copies only top-level object; nested objects are still shared. Deep copy (copy.deepcopy()) creates completely independent copy including nested objects.

Q: What is a generator in Python?

A: A function that yields values one at a time using 'yield'. Memory-efficient for large sequences. Example: def count(): while True: yield i (infinite sequence).

Q: What is a decorator?

A: A function that modifies another function's behavior without changing its code. Syntax: @decorator above function definition. Example: @staticmethod, @property.

Q: What is the difference between str.split() and str.partition()?

A: split() splits on all occurrences, returns list. partition() splits on first occurrence, returns 3-tuple: (before, separator, after).

Q: What does zip() do?

A: zip() pairs elements from multiple iterables: zip([1,2],[3,4]) → (1,3),(2,4). Stops at shortest iterable.

Q: What is enumerate()?

A: enumerate(iterable, start=0) adds a counter. Returns (index, value) pairs. Eliminates need for manual counter: for i, val in enumerate(lst).

Q: What are Python's membership operators?

A: 'in' — True if item found in sequence. 'not in' — True if item NOT found. Work on strings, lists, tuples, sets, dicts (checks keys).

Q: What is the difference between sort() and sorted()?

A: list.sort() modifies list in-place, returns None. sorted(iterable) returns a new sorted list without modifying original. sorted() works on any iterable.

Q: What is string formatting in Python?

A: Three methods: % formatting (old), str.format() (Python 3), f-strings (Python 3.6+, fastest). f'Hello {name}' is the modern recommended way.

Q: What is __str__ and __repr__?

A: __str__ returns human-readable string representation (used by print()). __repr__ returns unambiguous developer representation (used in REPL). Always define __repr__.

Q: What is the difference between input() in Python 2 and 3?

A: Python 3: input() always returns a string. Python 2: input() evaluated the expression (dangerous!); raw_input() was the safe string equivalent.

Q: What is a namespace in Python?

A: A namespace is a mapping from names to objects. Python has local, enclosing, global, and built-in namespaces (LEGB rule for name resolution).

Q: What is pickling?

A: Serializing Python objects to byte stream (pickle.dump()) for saving to file or sending over network. Unpickling (pickle.load()) restores the object.

Q: What is the GIL?

A: Global Interpreter Lock — a mutex in CPython that allows only one thread to execute Python bytecode at a time, limiting true multi-threading for CPU-bound tasks.

Q: What are Python's bitwise operators?

A: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift). Operate on individual bits of integers.

Q: What is the difference between is and == for None check?

A: Always use 'x is None' not 'x == None'. 'is' checks identity (only one None object exists). '==' could be overridden by __eq__ method.

Part B: Top 20 Python Interview Questions

Q: What is the difference between mutable and immutable types?

A: Mutable objects (list, dict, set) can be changed after creation. Immutable objects (int, str, tuple, frozenset) cannot be modified. Immutable objects are hashable and can be dictionary keys.

Q: Explain Python's memory management.

A: Python uses automatic memory management with reference counting and garbage collection (gc module). Objects are freed when reference count reaches 0. The garbage collector handles circular references.

Q: What is the difference between shallow and deep copy?

A: copy.copy() creates shallow copy — new container but same object references inside. copy.deepcopy() recursively copies all nested objects. For simple structures, slicing [:] or list() also creates shallow copies.

Q: What are Python decorators? Give a practical example.

A: Decorators are higher-order functions that modify another function. @staticmethod, @classmethod, @property are built-in decorators. Custom: def timer(func): def wrapper(*args): start=time.time(); result=func(*args); print(time.time()-start); return result — measures execution time.

Q: Explain list comprehension vs generator expression.

A: List comprehension [x**2 for x in range(n)] creates the entire list in memory immediately. Generator expression (x**2 for x in range(n)) creates values lazily (one at a time) using yield — memory-efficient for large datasets.

Q: What is the LEGB rule?

A: Python's name resolution order: Local (inside function) → Enclosing (outer function for nested) → Global (module level) → Built-in (Python builtins). Python searches in this order when resolving variable names.

Q: How does Python's 'is' operator differ from ==?

A: == checks VALUE equality (calls __eq__). 'is' checks IDENTITY (same object in memory, same id()). Small integers (-5 to 256) and short strings are cached by Python, so 'is' may return True unexpectedly for them.

Q: What is *args and **kwargs? When would you use them?

A: *args captures extra positional arguments as a tuple. **kwargs captures extra keyword arguments as a dict. Use when number of arguments is unknown: def log(*args, **kwargs) can accept any combination of arguments.

Q: Explain Python's exception hierarchy.

A: BaseException → Exception → (ArithmeticError, LookupError, ValueError, TypeError, IOError...). Use specific exceptions in except clauses. Catching Exception misses SystemExit/KeyboardInterrupt; BaseException catches everything.

Q: What is a context manager? How do you create one?

A: Object managing resources with __enter__ and __exit__ methods, used with 'with'. Custom: class MyCtx: def __enter__(self): ...; def __exit__(self,*e): .... Or use @contextmanager decorator from contextlib.

Q: What is the difference between staticmethod, classmethod, and instance method?

A: Instance method: receives 'self' (instance). Class method (@classmethod): receives 'cls' (class) — used for alternative constructors. Static method (@staticmethod): no implicit first parameter — utility functions.

Q: How does Python's garbage collection work?

A: Primary: reference counting (freed immediately when count=0). Secondary: cyclic garbage collector detects reference cycles (A→B→A). gc.collect() forces collection. gc module can configure generations (0,1,2).

Q: What are Python's built-in functions you use most often?

A: len(), range(), type(), isinstance(), print(), input(), int/float/str/list/dict/set (type constructors), zip(), enumerate(), map(), filter(), sorted(), sum(), max(), min(), abs(), round().

Q: Explain the difference between threading and multiprocessing.

A: threading: multiple threads in ONE process sharing memory (GIL limits CPU-bound performance; good for I/O-bound). multiprocessing: multiple separate processes (each has own GIL; true parallelism for CPU-bound tasks).

Q: What is a Python virtual environment?

A: An isolated Python installation for a project (python -m venv env). Allows different projects to have different package versions without conflicts. Activate: source env/bin/activate (Linux/Mac), env\Scripts\activate (Windows).

Q: What are Python data classes? (Python 3.7+)

A: @dataclass decorator auto-generates __init__, __repr__, __eq__ for classes storing data. from dataclasses import dataclass; @dataclass class Point: x:float; y:float — creates Point(x=1.0, y=2.0) with full boilerplate.

Q: How do you handle large files efficiently in Python?

A: Use 'with open() as f' and iterate line-by-line (for line in f: process(line)) — reads one line at a time, O(1) memory. For CSV: csv.reader. For JSON: ijson for streaming. For binary: chunks with f.read(chunk_size).

Q: What is the difference between append, extend, and insert for lists?

A: append(x) adds x as single element at end. extend(iterable) adds all elements of iterable at end. insert(i, x) inserts x at position i, shifting elements right. All modify in-place.

Q: Explain Python's 'yield' keyword.

A: yield pauses a generator function and returns a value. Next call resumes from after yield. Enables lazy evaluation: def infinite_counter(): n=0; while True: yield n; n+=1 — generates numbers on demand without storing all.

Q: What are type hints in Python?

A: Optional annotations indicating expected types (Python 3.5+). def greet(name: str) -> str: return 'Hi ' + name. Not enforced at runtime but used by IDEs and type checkers (mypy). Improves readability and catches errors early.

Part C: Common Errors Beginners Make

Error Type	Cause	Example	Fix
SyntaxError	Typo or missing syntax	if x > 5 print(x)	Add colon: if x > 5:
IndentationError	Wrong or mixed indentation	def f():\nprint('hi')	Indent print() with 4 spaces
NameError	Variable used before defined	print(x) # x not defined	Define x before using it
TypeError	Wrong type for operation	'5' + 5	Convert: int('5') + 5
ValueError	Right type, wrong value	int('hello')	Validate input before conversion
IndexError	List index out of range	lst[5] when len=3	Check index < len(lst)
KeyError	Dict key doesn't exist	d['key'] when key absent	Use d.get('key') or check 'key' in d
ZeroDivisionError	Dividing by zero	10 / 0	Check divisor != 0 before dividing
AttributeError	Object lacks attribute	'hello'.append('!')	.append() is for lists, not strings
ImportError	Module not found	import numpy	pip install numpy first
RecursionError	Infinite recursion	Missing base case in recursive fn	Add proper base case
FileNotFoundError	File path wrong	open('data.txt')	Check file exists and path is correct

Part D: Python Cheat Sheet

📌 Variables & Types

```
x = 10          # int
y = 3.14        # float
name = 'Alice'  # str
flag = True     # bool
z = None        # NoneType
type(x)         # → <class 'int'>
```

📌 Operators

```
+ - * /          # Arithmetic
// (floor div)  % (modulo)  ** (power)
== != > < >= <= # Comparison
and  or  not     # Logical
& | ^ ~ << >>    # Bitwise
in  not in  is   # Membership / Identity
```

📌 Strings

```
s = 'Hello'
s.upper()  s.lower()  s.strip()
s.replace('H','J')  s.split(',')
s[1:4]      # Slice
s[::-1]     # Reverse
f'Hi {name}' # f-string
```

📌 Lists

```
lst = [1, 2, 3]
lst.append(4)   lst.pop()
lst.insert(0,0) lst.remove(2)
lst.sort()      lst.reverse()
len(lst)  sum(lst)  max(lst)
[x**2 for x in lst]  # Comprehension
```

📌 Tuples & Sets

```
t = (1, 2, 3)    # Immutable
t[0]  len(t)     # Access
s = {1, 2, 3}    # Set (unique)
s.add(4)  s.remove(1)
s1 | s2   s1 & s2   s1 - s2
frozenset(s)     # Immutable set
```

📌 Dictionaries

```
d = {'a':1, 'b':2}
d['a']      d.get('c', 0)
d['c'] = 3  del d['a']
d.keys()  d.values()  d.items()
for k,v in d.items(): ...
{k:v for k,v in d.items() if v>1}
```

📌 Control Flow

```
if x>0: ...   elif x==0: ...   else: ...
for i in range(10): ...
while condition: ...
break    # Exit loop
continue # Skip iteration
pass     # Placeholder
```

📌 Functions

```
def greet(name, greeting='Hi'):
    return f'{greeting}, {name}'
greet('Alice')   greet('Bob','Hello')
def total(*args): return sum(args)
def info(**kwargs): print(kwargs)
sq = lambda x: x**2  # Anonymous fn
```

📌 OOP

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self): pass
class Dog(Animal):  # Inheritance
    def speak(self): return 'Woof!'
```

📌 File I/O

```
with open('f.txt','r') as f:
    content = f.read()
with open('f.txt','w') as f:
    f.write('Hello')
import json
json.dump(data,f)  json.load(f)
```

📌 Error Handling

```
try:
    result = 10 / x
except ZeroDivisionError:
    print('Cannot divide by zero')
except (TypeError, ValueError) as e:
    print(f'Error: {e}')
```

📌 Useful Modules

```
import math    # sqrt, pi, log, ceil
import random  # randint, choice, shuffle
import os      # path, getcwd, listdir
import sys     # argv, exit, path
import datetime # date, time, timedelta
import re      # Regular expressions
```

Part E: Practical Exam Preparation Guide

Before the Exam	During the Exam	Common Mistakes to Avoid
Practice all 30 programs manually	Read the question carefully twice	Forgetting colon after if/for/def
Memorize common syntax patterns	Write algorithm/flowchart first	Using = instead of == in conditions
Understand each program's logic	Desk-check with sample values	Off-by-one errors in range()
Practice typing code without IDE help	Comment your code clearly	Forgetting int(input()) conversion
Know all viva answers by heart	Test with boundary values (0,-1)	Indentation errors in nested blocks
Review time & space complexity	Handle edge cases explicitly	Missing return in functions
Prepare for surprise modifications	Format output neatly	Uninitialized accumulator variables

Part F: 30-Day Python Practice Roadmap

Week	Days	Topics	Practice Goal
Week 1 (Basics)	1-2	Python setup, print(), variables, data types	Q1-Q5: Hello World to Swap
	3-4	Operators, if-elif-else, comparison operators	Q6-Q8: Even/Odd, Largest numbers
	5-7	Math formulas, type conversion, formatting	Q9-Q10: SI, Percentage & Grade
Week 2 (Loops)	8-9	for loop, while loop, range()	Q11-Q13: Count, Sum, Table
	10-11	Accumulator pattern, nested loops	Q14-Q15: Factorial, Reverse
	12-14	Digit manipulation, modulus technique	Q16-Q18: Count/Sum digits, Palindrome
Week 3 (Logic)	15-16	Number theory: Armstrong, Prime	Q19-Q20: Armstrong, Prime check
	17-18	Series generation, Fibonacci	Q21-Q22: Fibonacci, Prime range
	19-21	Pattern printing, nested loops	Q23-Q25: Triangle, Pyramid, Diamond
Week 4 (Strings)	22-23	String methods, slicing, indexing	Q26-Q27: Reverse & Palindrome string
	24-25	String iteration, vowels/consonants	Q28-Q29: Count vowels, word count
	26-27	Dictionaries, frequency analysis	Q30: Character frequency
Week 5 (Projects)	28	Functions, modular programming	Mini Projects 1-2
	29	Loops, user interaction, game logic	Mini Projects 3-4
	30	Revision, mock viva, practice exam	Review all programs + full viva Q&A

🎓 Best of Luck for Your Exams! 🎓

"The secret of getting ahead is getting started." — Mark Twain

Practice consistently. Code daily. Ask questions. You've got this!